

Enterprise Knowledge Assistant

Anthrasync AI Engineer Assignment | Author: Kancharla Surya Teja

1. High-Level Architecture

The system is a Retrieval-Augmented Generation (RAG) pipeline split into two phases: an offline ingestion phase that builds a searchable index from source PDFs, and an online query phase that answers user questions against that index.

Ingestion phase:

- PDF Loader (PyMuPDF) extracts text and page numbers from each source document.
- Semantic Chunking (RecursiveCharacterTextSplitter, 900-character chunks, 150-character overlap) splits extracted text into retrieval-sized units while preserving paragraph/sentence boundaries where possible.
- Local Embeddings (BAAI/bge-base-en-v1.5) convert each chunk into a 768-dimensional vector without an external API call.
- ChromaDB persists vectors plus metadata (document name, page number, category, chunk ID) for later retrieval.

Query phase:

- Streamlit UI collects the user's question and displays the answer, confidence, latency, and source citations.
- FastAPI backend exposes POST /ask, GET /health, and auto-generated Swagger/ReDoc docs.
- MMR Retriever queries ChromaDB for the top 5 chunks, using Maximal Marginal Relevance to reduce redundant near-duplicate chunks in the result set.
- Conversation Memory injects prior turns from the same session so follow-up questions (e.g. “what about contractors?”) resolve correctly.
- Prompt Builder assembles retrieved chunks, memory, and the question into a grounding-constrained prompt that instructs the model to answer only from context and explicitly say so when information is unavailable.
- Groq LLM (Llama 3.3 70B Versatile) generates the answer, which is returned with source citations.

2. Data Flow

Ingestion (run once per document set, offline):

Enterprise PDFs → PDF Loader → Semantic Chunking → Local BGE Embeddings → ChromaDB

Query (per user request, online):

User question (Streamlit) → FastAPI /ask → MMR Retriever (ChromaDB) → Conversation Memory merge → Prompt Builder → Groq LLM → Answer + source citations → FastAPI response → Streamlit UI

A request never touches the LLM without first passing through retrieval — there is no path where the model answers from parameters alone. This is the mechanism behind the hallucination-prevention requirement: the model is structurally limited to the retrieved context plus conversation history, and the prompt additionally instructs it to decline when that context doesn't contain the answer.

3. Component Explanation

Component	Responsibility	Technology
PDF Loader	Extracts raw text and page numbers from source PDFs	PyMuPDF
Chunker	Splits text into overlapping, retrieval-sized segments	RecursiveCharacterTextSplitter
Embedder	Converts chunks into dense vectors, locally, no API cost	BAAI/bge-base-en-v1.5
Vector Store	Persists vectors and metadata; supports similarity search	ChromaDB
Retriever	Selects the top-K most relevant, least redundant chunks	MMR Search, Top K = 5
Conversation Memory	Tracks per-session history for follow-up questions	Session-scoped ConversationManager
Prompt Builder	Assembles context, memory, and question into a grounded prompt	Custom prompt template
LLM	Generates the final answer from retrieved context only	Groq, Llama 3.3 70B Versatile
API Layer	Exposes /ask, /health; validates requests/responses	FastAPI, Pydantic v2
Frontend	Question input, answer display, source citations, session state	Streamlit

4. Scalability Considerations

The current deployment is single-node and local, appropriate for this assignment's scope (22 documents, 671 chunks) but with identifiable ceilings if document volume or user load grows.

Storage and retrieval

ChromaDB currently runs embedded, in-process; at higher volume this would move to client-server mode (or a managed vector database) so the index scales independently of the API. Retrieval is dense-vector-only (MMR over BGE embeddings) — fine for natural-language questions, but exact-match content

(policy IDs, acronyms) falls outside what embeddings capture well, so hybrid search (BM25 + dense) would be the next addition as the corpus grows more heterogeneous.

Ingestion throughput

Embedding generation is local and CPU/GPU-bound. For hundreds to thousands of documents, ingestion would need to move from a synchronous script to a queue-based batch pipeline so re-indexing doesn't block the running application.

Concurrency and session state

FastAPI/Uvicorn can serve concurrent requests, but conversation memory is currently session-scoped in-process. Scaling the API horizontally (multiple instances behind a load balancer) would require moving session state into a shared store (e.g. Redis), and a re-ranking step before generation would let retrieval breadth grow without proportionally inflating prompt size and LLM latency.